

10Pearls

Data Science Engineering Internship

AeroWatch

An End to End Data Engineering and Machine Learning System for Air Quality
Forecasting in Pakistan

Project Documentation

Prepared by Hussain Ali

Final Year, Data Engineering

August 2026

Introduction

Air pollution is one of the most pressing environmental and public health challenges facing Pakistan today, with cities such as Lahore, Karachi, Islamabad and Peshawar regularly experiencing air quality levels that pose a genuine risk to residents. This project, developed as part of a Data Science Engineering internship at 10Pearls, sets out to build a complete, working system that can observe current air quality, learn from historical patterns, and forecast what the air will look like over the coming three days for each of these four cities.

Rather than treating this as a single machine learning exercise, the project was approached as a full lifecycle system, the kind that a data engineering team would actually be expected to build and maintain in a real organisation. That meant designing a pipeline to collect live data every hour, a separate pipeline to retrain and evaluate models every day, a place to store both the data and the models in a structured and versioned way, and a dashboard that a non technical person could open and immediately understand. The result is a project named AeroWatch, which is described in detail throughout this document.

Objectives of the Project

The objectives set for this project were to build an automated pipeline that collects live air quality and weather readings for Lahore, Islamabad, Karachi and Peshawar every hour, to gather a full year of historical data so that the machine learning models would have enough examples to learn from, to train and compare several different forecasting models rather than relying on a single approach, to store both the processed data and the trained models in a proper feature store and model registry rather than scattered files, to orchestrate the entire pipeline so that it runs on its own schedule without manual intervention, and to present the results through an interactive dashboard that shows not just a forecast but also why the model produced that forecast and how accurate it has actually been in practice.

System Architecture

The system is built around four cooperating parts. The first is a feature pipeline that reaches out to two live data providers, the AQICN air quality service and the OpenWeather service, once every hour for each of the four cities. The readings it collects are cleaned, combined with time based features such as the hour and day of the week, and combined with a short history of recent readings to calculate values such as yesterday's air quality index and a twenty four hour rolling average. The result is written into the Hopsworks Feature Store, which acts as the single source of truth for both training and prediction.

The second part is a training pipeline that runs once a day. It reads the entire history of features collected so far, builds three separate forecasting targets representing the air quality twenty four, forty eight and seventy two hours ahead, and trains three different kinds of model on the same data, a linear Ridge Regression model, a Random Forest ensemble, and a small neural network built with

TensorFlow and Keras. Each model is scored on how accurately it predicts the three future time horizons, and the best performing model is registered in the Hopsworks Model Registry.

The third part is the orchestration layer, built with Apache Airflow running inside Docker. Two scheduled workflows drive the system, one that triggers the feature pipeline every hour and one that triggers the training pipeline every day, along with a third workflow that logs each day's forecasts so that they can later be checked against what actually happened.

The fourth part is the AeroWatch dashboard, a Streamlit application that loads the latest model and the latest readings from Hopsworks and presents them to the user. The diagram below shows how these four parts connect to one another.

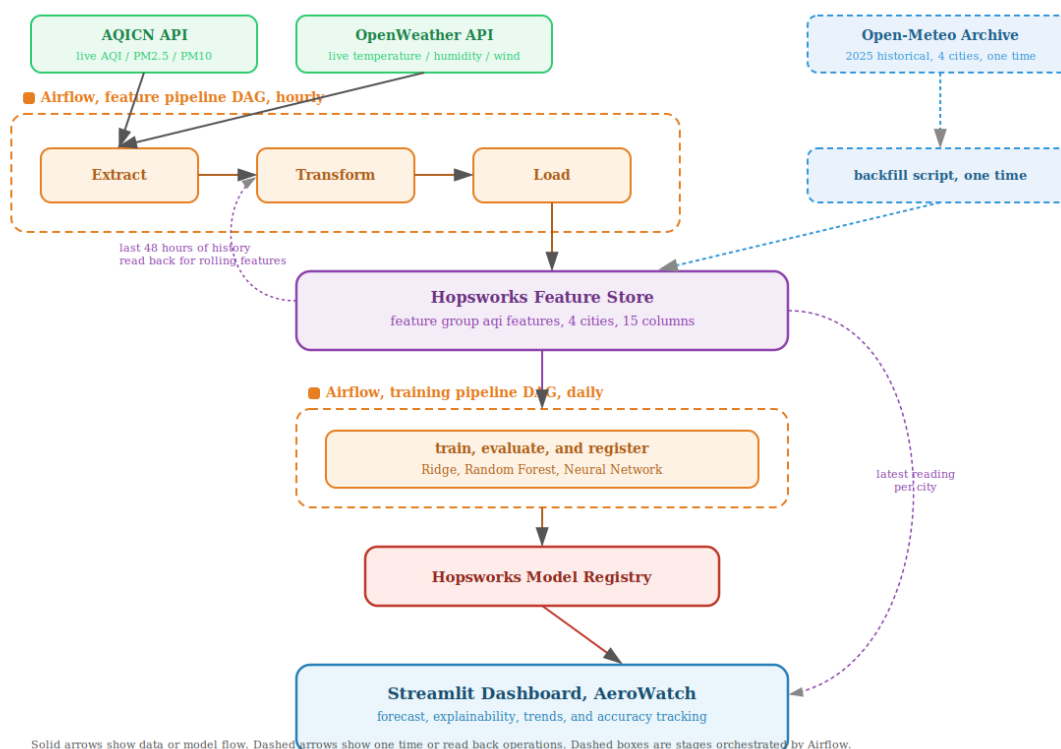


Figure 1. System architecture of the AeroWatch pipeline.

Data Sources and Historical Backfill

Two different kinds of data sources were used in this project, live sources for ongoing operation and a historical source for training data. The live sources are the AQICN API, which provides the current air quality index along with particulate matter readings, and the OpenWeather API, which provides temperature, humidity and wind speed. Both are free services that report only the current conditions, which is appropriate for the hourly pipeline but not useful on their own for building a training dataset, since a machine learning model needs thousands of past examples rather than a single current reading.

To solve this, a full year of historical data for 2025 was collected separately for all four cities using the Open Meteo archive, a free service that provides both historical weather and historical air quality reconstructions without requiring an account or an API key. This backfill script was run once to populate the feature store with tens of thousands of hourly records before any model was trained, giving the training pipeline a genuine history to learn seasonal and daily patterns from. It is worth noting honestly that AQICN's air quality index and Open Meteo's reconstructed index are calculated using slightly different methodologies, which is a reasonable and disclosed limitation of combining a live source with a historical one.

The Feature Pipeline

The feature pipeline is responsible for turning a raw API reading into something a model can learn from. When it runs, it fetches the current readings for a city, fills in any missing pollutant values, and calculates two kinds of engineered feature. The first are simple time features, the hour of day, the day of the month, the month, and whether the reading falls on a weekend. The second are lag and rolling features, calculated by reading back the last forty eight hours of that same city's history from the feature store, which allows the pipeline to compute what the air quality index was exactly twenty four hours earlier, how much it has changed since the previous reading, and what its twenty four hour rolling average has been. These history aware features turned out to be some of the most useful inputs for the forecasting models.

The pipeline processes all four cities independently using Airflow's dynamic task mapping feature, so a fifth city could be added to the system in future without any change to the orchestration logic itself, only to a single list of city names.

Training Pipeline and Model Selection

Rather than committing to one modelling approach, the training pipeline was designed to experiment with three models that span a meaningful range, from a simple statistical baseline through to a deep learning model. Ridge Regression represents the statistical end of that range, a well understood linear model that is fast to train and easy to reason about. Random Forest represents a classical ensemble approach that can capture non linear relationships between the weather readings and the resulting air quality. A small neural network built with Keras represents the deep learning end of the spectrum.

Because the goal is to forecast three separate points in the future rather than just the next reading, the models are trained to predict all three horizons, twenty four, forty eight and seventy two hours ahead, from a single set of inputs describing conditions right now. This is done by looking ahead in each city's own historical record to find what the air quality actually was at each of those future points, and using that as the training target. In practice, Random Forest has consistently outperformed the other two models across every horizon, achieving an R squared of around zero point nine two for the twenty four hour forecast, meaning it explains roughly ninety two percent of the variation in air quality one day ahead, with accuracy naturally decreasing a little for the more distant forty eight and seventy two hour forecasts. Ridge Regression trails behind both other

models, which is expected given the genuinely non linear relationship between pollutant levels, weather conditions and air quality outcomes.

The Model Registry

Every time the training pipeline runs, the winning model is saved into the Hopsworks Model Registry along with the metrics it achieved on each forecast horizon. Each save creates a new version rather than overwriting the previous one, so earlier models remain available and the system could be rolled back to a previous version if a future retrain ever performed worse. Because the three candidate models are built on two different underlying frameworks, ordinary machine learning libraries for Ridge and Random Forest and TensorFlow for the neural network, the registry code was written to detect which framework produced the winning model and save it in the correct format automatically, so the rest of the system never needs to know or care which of the three models actually won on a given day.

Orchestration with Apache Airflow

Apache Airflow was chosen to orchestrate the whole system because it is the tool most commonly used in industry for exactly this kind of scheduled, dependency aware pipeline. It runs inside Docker alongside a Postgres database, and the whole environment is defined so that anyone could bring the system up on their own machine with a single command. Three separate workflows are defined. The feature pipeline workflow runs every hour and processes all four cities. The training pipeline workflow runs once a day and retrains all three candidate models before registering the best one. A third workflow, the forecast logging pipeline, also runs once a day and records what the currently registered model is predicting for each city, so that those predictions can later be compared against what actually happened.

The AeroWatch Dashboard

The dashboard, named AeroWatch, is the part of the system a person actually interacts with. It is built with Streamlit and organised into four sections. An overview page shows today's actual air quality for all four cities at a glance, with a colour coded card for each city using the standard air quality categories from good through to hazardous. A city forecast page shows a single city's three day forecast in detail, alongside a trend chart and, on request, an explanation of why the model produced that particular forecast. An exploratory analysis page shows historical trends, including how air quality varies by hour of day and by month, and how the different measured readings correlate with one another. A forecast accuracy page compares what the model predicted in the past against what actually happened once enough time has passed for that comparison to be possible.

Care was taken to make the dashboard genuinely usable rather than simply functional. Expensive operations, such as loading a full year of historical data or calculating an explanation for a specific prediction, are only performed when the relevant section is opened or a button is explicitly pressed, rather than being recalculated every time the page refreshes. This keeps the dashboard responsive even though the underlying dataset is large.

Model Explainability with SHAP

A forecast on its own is not always convincing. If the dashboard predicts an air quality index of one hundred and seventy for tomorrow, a reasonable question to ask is why. To answer that question, the project uses SHAP, a widely used method for explaining individual predictions made by a machine learning model. For each city, the dashboard can calculate which of the input features, such as today's particulate matter reading, the current wind speed, or the recent rolling average, pushed that particular forecast higher or lower, and by roughly how much. This is shown as a simple bar chart, with features that increase the predicted air quality index shown in red and features that decrease it shown in green.

Hazard Alerts

Beyond simply reporting a number, the dashboard actively watches for genuinely concerning forecasts. If any of the three forecast days for a city crosses into the unhealthy range or worse, a clear warning banner appears on that city's page, and a summary warning also appears on the overview page listing every city currently at risk over the coming three days. This turns the dashboard from a passive reporting tool into something closer to an early warning system.

Forecast Accuracy Tracking

One of the more distinctive parts of this project is that it does not only rely on accuracy figures measured during training. Every day, the currently registered model's predictions for all four cities are logged, along with which model made them. Once enough real time has passed for those predicted dates to actually arrive, the system automatically compares the prediction against the real reading that eventually came in, and calculates the true, live error. This is meaningfully stronger evidence than a test set score calculated once during training, because it reflects how the model is actually performing against reality, day after day, and because every logged prediction is tagged with which specific model made it, so if the neural network or Ridge Regression were ever to outperform Random Forest on a future day, that improvement would show up here without any further changes to the code.

Deployment

The AeroWatch dashboard is deployed publicly using Streamlit Community Cloud, so that it can be viewed by anyone with the link rather than requiring the underlying pipeline to be running on a particular machine. The feature pipeline, training pipeline and forecast logging pipeline continue to run through Apache Airflow, orchestrated in Docker, which is the appropriate way to run scheduled background jobs. Deploying only the dashboard publicly, while keeping the scheduled pipelines running through Airflow, is a deliberate and reasonable division of responsibilities for a project of this scale.

Use Case Diagram

The diagram below summarises how the different people and automated processes interact with the system. A dashboard user, representing anyone viewing the AeroWatch dashboard, can view current air quality, explore forecasts, review historical trends, inspect why a forecast was made, and see how accurate past forecasts have been. A data engineer is responsible for running and maintaining the underlying pipelines. The Airflow scheduler itself is shown as an actor because it triggers the feature pipeline, training pipeline and forecast logging pipeline automatically, without a person needing to start them by hand.

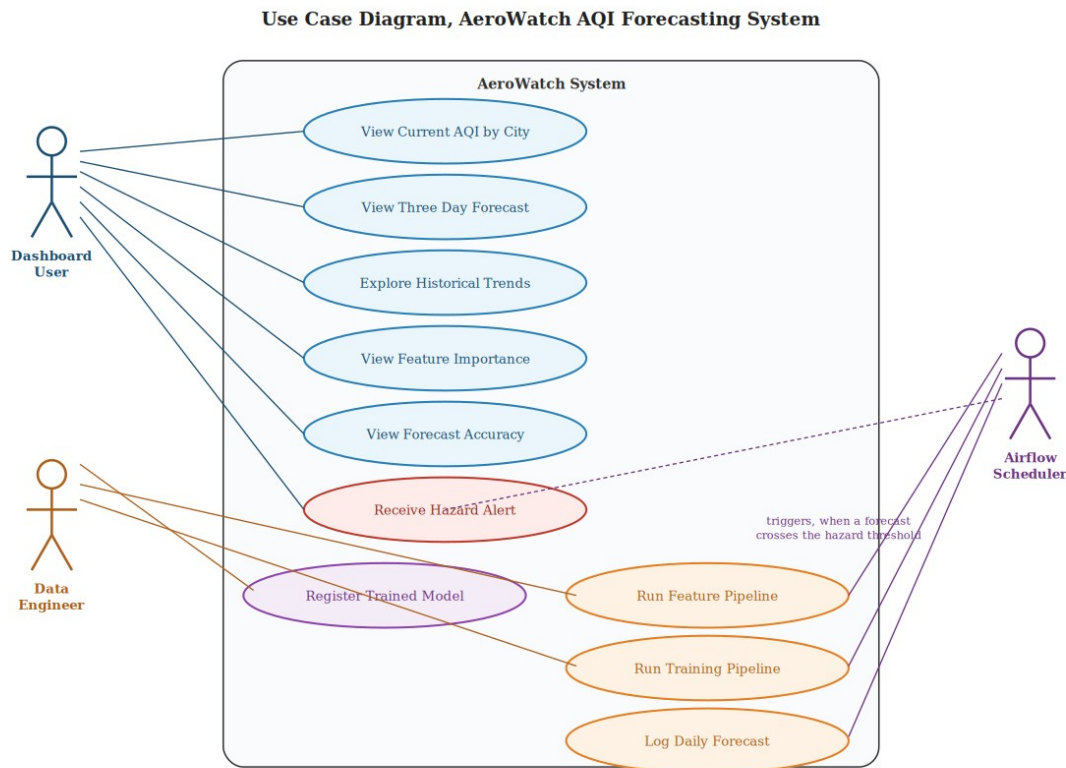


Figure 2. Use case diagram showing how users and the scheduler interact with the system.

Results and Model Performance

Taking Random Forest as the model that has consistently been selected by the training pipeline, its performance across the three forecast horizons demonstrates that a genuinely useful multi day air quality forecast is achievable from a reasonably compact set of features. The twenty four hour forecast achieves the strongest performance, which is expected since the near future is inherently easier to predict than three days out, and the gradual decline in accuracy at forty eight and seventy two hours reflects a realistic and honestly reported limitation rather than something to be hidden.

Challenges and How They Were Solved

Several genuine engineering challenges came up over the course of building this system, and it is worth documenting them honestly rather than presenting the project as though everything worked on the first attempt.

- Getting the Airflow Docker image to build required adding a C compiler to the container, since one of Hopsworks's own dependencies needed to compile a small library from source.
- Uploading a large, fully grown Random Forest model to the model registry repeatedly failed over an unreliable connection, until the model itself was made smaller by limiting its depth and number of trees, and the upload logic was given the ability to retry automatically.
- Deploying the dashboard publicly surfaced a subtle inefficiency, where every page load was unintentionally importing TensorFlow even when the currently active model did not need it, which was fixed by deferring that import until it is genuinely required.
- Running four cities concurrently inside a memory constrained container occasionally caused a task to be killed by the operating system before Airflow could record why, which was resolved by limiting how many of the four cities' tasks run at the same time.

Future Work

A small number of extensions were identified but deliberately left for future iterations rather than rushed into this version. A dedicated Flask or FastAPI service could expose predictions programmatically for other systems to consume, rather than only through the dashboard. Automated alerts could be delivered by email or messaging service rather than only being visible to someone who opens the dashboard. A continuous integration workflow using GitHub Actions could run automated checks on the code itself whenever a change is pushed, alongside the existing Airflow automation of the pipelines.

Conclusion

AeroWatch demonstrates a complete, working example of a modern data engineering and machine learning system, taken from raw API data through to a public, explainable, continuously validated forecasting dashboard. Every stage of the system, from hourly ingestion through to daily retraining, model registration, orchestration and public deployment, was built, tested against real data, and in several cases debugged against genuine failures encountered along the way. The result is a project that reflects not just an ability to train a machine learning model, but an understanding of what it takes to keep one running reliably over time.